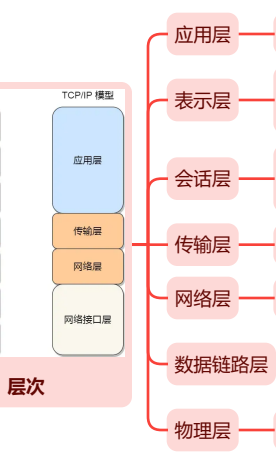
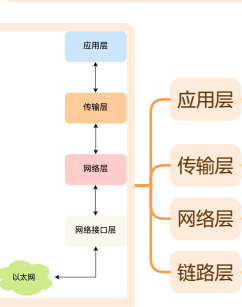
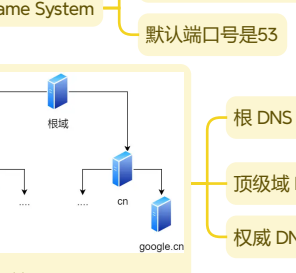
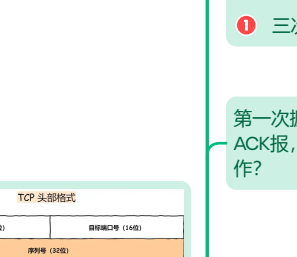
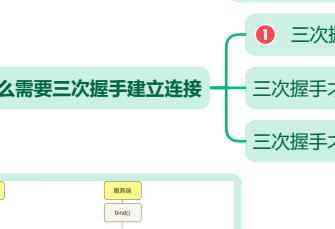
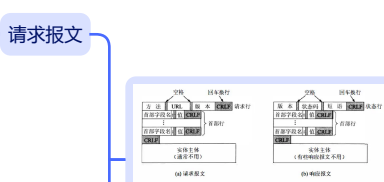
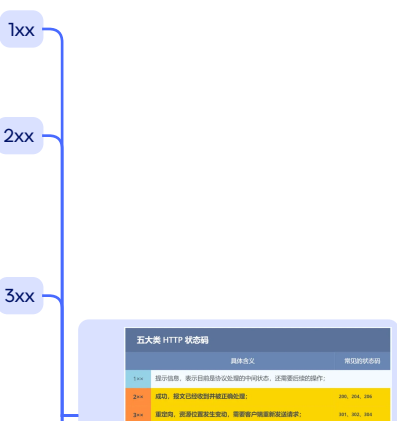
8. 网络篇

- ❗ OSI七层模型
  - 定义 — 是国**际标准化组织**（ISO）提出的网络通信模型 — 它将网络通信划分为**七个层次**，每一层都有**特定**的功能，旨在**解决不同层面的网络通信**问题
  - 

    层次
    - 应用层 — 提供应用程序与网络通信的接口 — 例如 HTTP、FTP、SMTP 等协议
    - 表示层 — 负责数据格式转换、加密/解密、压缩/解压缩，确保不同系统能正确解析数据
    - 会话层 — 负责建立、管理和终止会话，确保通信过程的同步和恢复能力 — 如 RPC、SQL 会话管理
    - 传输层 — 负责端到端的数据传输 — 如 TCP（可靠）和 UDP（不可靠）
    - 网络层 — 负责数据的路由、转发、分片 — 如 IP、ICMP、ARP
    - 数据链路层 — 负责数据封装成帧，进行 MAC 地址寻址、差错检测和纠正 — 如 以太网、PPP
    - 物理层 — 负责在物理网络中传输数据帧 — 如 网线、Wi-Fi、光纤
- ❗ TCP/IP模型
  - 定义 — 是互联网**实际应用**的网络通信模型 — 将网络通信划分为 四个层次，用于描述数据如何在不同设备间传输
  - 
    层次
    - 应用层 — 提供网络服务，支持 HTTP、SMTP、FTP、DNS 等应用程序
    - 传输层 — 处理主机到主机的通信 — TCP、UDP
    - 网络层 — 负责 IP 地址寻址、路由选择和数据包转发 — IP、ICMP、ARP
    - 链路层 — 通过有线或无线介质传输比特流 — 以太网、Wi-Fi、PPP
- ❗ DNS
  - Domain Name System
    - 是互联网中用于**将域名转换为对应IP地址的分布式数据库系统** — 在域名中，越靠右的位置表示其层级越高
    - 默认端口号是53
  - 
    域名的层级关系
    - 根 DNS 服务器（.） — 根域的 **DNS 服务器信息**保存在互联网中**所有的** DNS 服务器中 — 任何 DNS 服务器就都可以找到并访问根域 DNS 服务器
    - 顶级域 DNS 服务器（.com）
    - 权威 DNS 服务器（server.com）
  - DNS 域名解析的工作流程
    - 客户端首先会发出一个 DNS 请求 — 问 www.server.com 的 IP 是啥 — 并发给本地 DNS 服务器
    - 本地域名服务器收到客户端的请求后
      - 如果缓存里的表格能找到 www.server.com — 则它直接返回 IP 地址
      - 如果没有 — 本地 DNS 会去问它的根域名服务器
    - 根 DNS 收到来自本地 DNS 的请求后 — 发现后置是 .com — .com 顶级域名服务器地址给你，你去问问它
    - 本地 DNS 收到.com顶级域名服务器的地址后 — 发起请求问"老二， 你能告诉我 www.server.com 的 IP 地址吗?
    - 顶级域名服务器说 — 我给你负责 www.server.com 区域的权威 DNS 服务器的地址
    - 本地 DNS 于是转向问权威 DNS 服务器 — 老三，www.server.com对应的IP是啥呀 — server.com 的权威 DNS 服务器，它是域名解析结果的原出处
    - 权威 DNS 服务器查询后将对应的 IP 地址 X.X.X.X 告诉本地 DNS
    - 本地 DNS 再将 IP 地址返回客户端，客户端和目标建立连接
  - DNS的底层
    - 基于UDP协议实现 — 使用UDP协议进行**域名解析**和**数据传输**
    - 原因
      - 低延迟 — UDP是一种无连接的协议 — 不需要在数据传输前建立连接 — 可以减少传输时延
      - 简单快速 — UDP相比于TCP更简单，没有TCP的连接管理和流量控制机制，传输效率更高
      - 轻量级 — UDP头部较小 — 占用较少的网络资源，对于小型请求和响应来说更加轻量级
- 传输层
  - 

    tcp
    - 头部
      - 序列号
        - 在建立连接时由计算机生成的随机数作为其初始值
        - 通过 SYN 包传给接收端主机，每发送一次数据，就「累加」一次该「数据字节数」的大小 — 用来解决网络包乱序问题
      - 确认应答号 — 指下一次「期望」收到的数据的序列号 — 发送端收到这个确认应答以后可以认为在这个序号以前的数据都已经被正常接收 — 用来解决丢包的问题
      - 控制位
        - ACK — 该位为 1 时，「确认应答」的字段变为有效 — TCP 规定除了最初建立连接时的 SYN 包之外该位必须设置为 1
        - RST — 该位为 1 时，表示 TCP 连接中出现异常必须强制断开连接
        - SYN — 该位为 1 时，表示希望建立连接 — 并在其「序列号」的字段进行序列号初始值的设定
        - FIN — 该位为 1 时，表示今后不会再有数据发送，希望断开连接
    - ❗ 三次握手过程
      - 第一次
        - 客户端会随机初始化序号
        - 同时把 SYN 标志位置为 1，表示 SYN 报文
        - 之后客户端处于 SYN-SENT 状态
      - 第二次
        - 服务端收到客户端的 SYN 报文后，首先服务端也随机初始化自己的序号
        - 其次把 TCP 首部的「确认应答号」字段填入 client_isn + 1
        - 接着把 SYN 和 ACK 标志位置为 1
        - 之后服务端处于 SYN-RCVD 状态
      - 第三次
        - 客户端收到服务端报文后，还要向服务端回应最后一个应答报文
        - ACK 标志位置为 1
        - 其次「确认应答号」字段填入 server_isn + 1
        - 最后把报文发送给服务端，这次报文可以携带客户到服务端的数据，之后客户端处于 ESTABLISHED 状态
    - **tcp为什么需要三次握手建立连接**
      - ❗ 三次握手才可以阻止重复历史连接的初始化 — 防止旧的重复连接初始化造成混乱
      - 三次握手才可以同步双方的初始序列号
      - 三次握手才可以避免资源浪费
    - 
      ❗ 三次握手和 accept 是什么关系 — tcp 完成三次握手后，连接会被保存到内核的全连接队列 — 调用 accpet 就是从把连接取出来给用户程序使用
    - 第一次握手，客户端发送SYN报后，服务端回复ACK报，那这个过程中服务端内部做了哪些工作?
      - 服务端收到客户端发起的 SYN 请求后，内核会把该连接存储到**半连接队列**
      - 服务端收到**第三次握手**的 ACK 后，内核会把连接从半连接队列移除，**然后创建新的完全的连接，并将其添加到 accept 队列**，等待进程调用 accept 函数时把连接取出来
    - 大量SYN包发送给服务端服务端会发生什么事情?
      - 有可能会导致TCP 半连接队列打满，这样当 TCP 半连接队列满了，后续再在收到 SYN 报文就会丢弃，导致客户端无法和服务端建立连接
      - 避免 SYN 攻击方式，可以有以下四种方法
        - 调大 netdev_max_backlog — 当网卡接收数据包的速度大于内核处理的速度时，会有一个队列保存这些数据包 — 将队列调大，保存更多的数据包
        - 增大 TCP 半连接队列
        - 开启 tcp_syncookies — 开启 syncookies 功能就可以在不使用 SYN 半连接队列的情况下成功建立连接
        - 减少 SYN+ACK 重传次数 — 当服务端受到 SYN 攻击时，就会有大量处于 SYN_REVC 状态的 TCP 连接，处于这个状态的 TCP 会重传 SYN+ACK ，当重传超过次数达到上限后，就会断开连接
    -
      ❗ **TCP 四次挥手**
      - 客户端主动调用关闭连接的函数，于是就会发送 FIN 报文 — 这个 FIN 报文代表客户端不会再发送数据了，进入 **FIN_WAIT_1 状态**
      - 服务端收到了 FIN 报文，然后马上回复一个 ACK 确认报文
        - 此时服务端进入 **CLOSE_WAIT 状态**
        - TCP 协议栈会为 FIN 包插入一个文件结束符 EOF 到接收缓冲区中 — 服务端应用程序可以通过 read 调用来感知这个 FIN 包 — 这个 EOF 会被放在已排队等候的其他已接收的数据之后，所以必须要得继续 read 接收缓冲区已接收的数据
        - 客户端进入 **FIN_WAIT_2 状态**
      - 当服务端在 read 数据的时候，最后自然就会读到 EOF，接着 read() 就会返回 0
        - 这时服务端应用程序如果有数据要发送的话，就发完数据后才调用关闭连接的函数
        - 如果服务端应用程序没有数据要发送的话，可以直接调用关闭连接的函数
        - 这时服务端就会发一个 FIN 包，这个 FIN 报文代表服务端不会再发送数据了，之后处于 **LAST_ACK 状态**
      - 客户端接收到服务端的 FIN 包，并发送 ACK 确认包给服务端，此时客户端将进入 **TIME_WAIT 状态**
      - 服务端收到 ACK 确认包后，就进入了最后的 CLOSE 状态
      - 客户端经过 2MSL 时间之后，也进入 CLOSE 状态
    - 为什么4次握手中间两次不能变成一次? — 服务端应用程序**可能还有数据要发送**，所以并不能马上发送 FIN 报文，而是将发送 FIN 报文的控制权交给服务端应用程序
      - 如果服务端应用程序有数据要发送的话，就发完数据后，才调用关闭连接的函数
      - 如果服务端应用程序没有数据要发送的话，可以直接调用关闭连接的函数
    - 第三次挥手一直没发，会发生什么 — 当主动方收到 ACK 报文后，会处于 FIN_WAIT2 状态 — 就表示主动方的发送通道已经关闭，接下来将等待对方发送 FIN 报文，关闭对方的发送通道
      - 如果连接是用 shutdown 函数关闭的 — 连接可以一直处于 FIN_WAIT2 状态，因为它可能还可以发送或接收数据
      - close 函数关闭的孤儿连接 — 由于无法再发送和接收数据，所以这个状态不可以持续太久 — 如果在 60 秒后还没有收到 FIN 报文，连接就会直接关闭
    - 为什么四次挥手之后要等2MSL?
      - MSL 是 Maximum Segment Lifetime，报文最大生存时间
      - 2MSL 主要是为了确保网络中的所有数据包都已消失，防止延迟数据包造成干扰，避免因旧的连接信息干扰新的连接，确保连接的完全断开和稳定
    - 什么场景下服务端会主动断开连接呢
      - HTTP 没有使用长连接
      - HTTP 长连接超时
      - HTTP 长连接的**请求数量达到上限**
  - ❗ TCP和UDP
    - 连接
      - TCP — 面向连接的传输层协议，传输数据前先要**建立连接**
      - UDP — 不需要连接，即刻传输数据
    - 服务对象
      - TCP — 一对一的两点服务
      - UDP — 支持一对一、一对多、多对多的交互通信
    - 可靠性
      - TCP — **可靠交付数据**的，数据可以无差错、不丢失、不重复、按序到达
      - UDP — **尽最大努力交付**，不保证可靠交付数据
    - 拥塞控制、流量控制
      - TCP — 有拥塞控制和流量控制机制，保证数据传输的安全性
      - UDP — UDP 则没有，即使网络非常拥堵了，也不会影响 UDP 的发送速率
    - 首部开销
      - TCP — 首部长度较长，会有一定的开销
      - UDP — 首部只有 8 个字节，并且是固定不变的，开销较小
    - 传输方式
      - TCP — 流式传输，没有边界，但保证顺序和可靠
      - UDP — 一个包一个包的发送，是有边界的，但可能会丢包和乱序
  - TCP为什么可靠传输
    - 连接管理
      - 三次握手
      - 四次挥手
      - 连接管理机制能够建立起可靠的连接
    - 序列号 — TCP将每个字节的数据都进行了编号，这就是序列号
      - 能够保证可靠性，既能防止数据丢失，又能避免数据重复
      - 能够保证有序性，按照序列号顺序进行数据包还原
      - 能够提高效率，基于序列号可实现多次发送，一次确认
    - 确认应答 — 接收方接收数据之后，会回传ACK报文 — 报文中带有此次确认的序列号，用于告知发送方此次接收数据的情况
    - 超时重传
      - 在指定时间后，若发送端仍未收到确认应答，就会启动超时重传，向接收端重新发送数据包
      - 当接收端收到重复数据(通过序列号进行识别)时将其丢弃，并重新回传ACK报文
    - 流量控制 — TCP支持根据接收端的处理能力，来决定发送端的发送速度。 — 流量控制是通过在TCP报文段首部维护一个滑动窗口来实现的。
    - 拥塞控制 — 当网络拥堵严重时，发送端减少数据发送 — 拥塞控制是通过发送端维护一个拥塞窗口来实现
  - ❗ tcp粘包怎么解决
    - 粘包的问题 — 是因为不知道一个用户消息的边界在哪 — 如果知道了边界在哪，接收方就可以通过边界来划分出有效的用户消息
    - 分包的方式
      - 固定长度的消息 — 即每个用户消息都是固定长度的
      - 特殊字符作为边界 — 我们可以在两个用户消息之间插入一个特殊的字符串，这样接收方在接收数据时，读到了这个特殊字符，就把认为已经读完一个完整的消息
      - 自定义消息结构 — 我们可以自定义一个消息结构，由包头和数据组成，其中包头包是固定大小的，而且包头里有一个字段来说明紧随其后的数据有多大
  - ❗ TCP的拥塞控制
    - 目的 — 避免「发送方」的数据填满整个网络
    - 拥塞窗口
      - 为了在「发送方」调节所要发送数据的量，定义了一个叫做「拥塞窗口」的概念
      - 拥塞窗口 cwnd是发送方维护的一个的状态变量，它会根据网络的**拥塞程度动态变化**的
        - 发送窗口 swnd 和接收窗口 rwnd 是约等于的关系
        - 发送窗口的值是swnd = min(cwnd, rwnd)，也就是**拥塞窗口和接收窗口中的最小值**
      - 变化的规则
        - 只要网络中没有出现拥塞，cwnd 就会增大
        - 但网络中出现了拥塞，cwnd 就减少 — 发生了超时重传，就会认为网络出现了拥塞
      - 拥塞控制主要是四个算法
        - 慢启动
          - TCP 在刚建立连接完成后，首先是有个慢启动的过程 — 一点一点的提高发送数据包的数量，如果一上来就发大量的数据，这不是给网络添堵吗
          - **当发送方每收到一个 ACK，拥塞窗口 cwnd 的大小就会加 1** — 指数增长
        - 拥塞避免 — 每当收到一个 ACK 时，cwnd 增加 1/cwnd — 线性增长
        - 拥塞发生
          - 当发生了「超时重传」
            - ssthresh 设为 cwnd/2
            - cwnd 重置为 初始化值
          - 发生快速重传
            - 当接收方发现丢了一个中间包的时候 — 发送三次前一个包的 ACK，于是发送端就会快速地重传，不必等待超时再重传
            - TCP 认为这种情况不严重，因为大部分没丢，只丢了一小部分
              - ssthresh = cwnd
              - cwnd = cwnd/2
              - 进入快速恢复算法
        - 快速恢复
          - 拥塞窗口 cwnd = ssthresh + 3
          - 重传丢失的数据包
          - 如果再收到重复的 ACK，那么 cwnd 增加 1
          - 如果收到新数据的 ACK 后，把 cwnd 设置为第一步中的 ssthresh 的值
      - 慢启动门限 ssthresh （slow start threshold）状态变量
        - 当 cwnd < ssthresh 时 — 使用慢启动算法
        - 当 cwnd >= ssthresh 时 — 就会使用「拥塞避免算法」
        - 一般来说 ssthresh 的大小是 65535 字节
  - 怎么用udp实现http
    - 基于 UDP 的 **QUIC 协议** 可以实现类似 TCP 的**可靠性传输** — 在**http3** 就用了 quic 协议
    - 连接迁移 — QUIC支持在网络变化时快速迁移连接 — 例如从WiFi切换到移动数据网络，以保持连接的可靠性。
    - 重传机制 — QUIC使用重传机制来确保丢失的数据包能够被重新发送，从而提高数据传输的可靠性
    - 前向纠错 — QUIC可以使用前向纠错技术，在接收端修复部分丢失的数据，降低重传的需求，提高可靠性和传输效率
    - 拥塞控制 — QUIC内置了拥塞控制机制，可以根据网络状况动态调整数据传输速率，以避免网络拥塞和丢包，提高可靠性
- 应用层
  - 应用层有哪些协议 — HTTP、HTTPS、CDN、DNS、FTP
  - HTTP
    - 
      HTTP报文
      - 请求报文
        - 请求行 — 包含 请求方法、URL、HTTP 版本 — GET /index.html HTTP/1.1
        - 请求头部 — 包含关于请求的附加信息 — 如 Host、User-Agent、Accept 等 — Host: www.example.com
          User-Agent: Mozilla/5.0
          Accept: text/html,application/xhtml+xml
        - 空行 — 请求头结束后有一个空行，表示接下来是请求体
        - 请求体 — 可选，包含请求的数据 — 只有 POST、PUT 等方法才会有请求体 — 例如提交表单数据 — username=admin&password=123456
      - 响应报文
        - 状态行 — 包含 HTTP 版本、状态码、状态描述 — HTTP/1.1 200 OK
        - 响应头部 — 包含关于响应的附加信息 — 如 Content-Type、Content-Length、Server — Content-Type: text/html; charset=UTF-8
          Content-Length: 1024
          Server: nginx
        - 空行 — 头部结束后有一个空行
        - 响应体 — 服务器返回的实际数据 — 如 HTML 页面、JSON 数据
    - 
      ❗ HTTP常用的状态码
      - 1xx — 属于提示信息 — 是协议处理中的一种中间状态
      - 2xx — 成功 — 服务器成功处理了客户端的请求
        - 200 OK — 请求成功
        - 201 Created — 资源创建成功
        - 204 No Content — 请求成功，但无返回内容
      - 3xx — 重定向 — 客户端请求的资源发生了变动，需要客户端用新的 URL 重新发送请求获取资源
        - 304 Not Modified — 资源未修改，使用缓存
        - ❗ 301 和 302
          - 301 Moved Permanently — 资源**永久重定向**，浏览器会记住并更新地址 — 说明请求的资源已经不存在了，需改用新的 URL 再次访问
          - 302 Found — 资源**临时重定向**，浏览器每次都会重新请求 — 说明请求的资源还在，但暂时需要用另一个 URL 来访问
          - 301 和 302 都会在**响应头**里使用字段 **Location** — 指明后续要跳转的 URL，浏览器会自动重定向新的 URL
      - 4xx — 客户端错误 — 客户端发送的报文有误，服务器无法处理
        - 400 Bad Request — 请求格式错误，服务器无法解析
        - 401 Unauthorized — **未授权**，需身份验证
        - 403 Forbidden — 服务器拒绝请求（权限不足）
        - 404 Not Found — **请求的资源不存在**
        - 405 Method Not Allowed — **请求的方法类型不支持** — 如 POST 访问只支持 GET 的接口
      - 5xx — 服务器错误 — 服务器处理时内部发生了错误
        - 500 Internal Server Error — **服务器内部错误**
        - 503 Service Unavailable — 服务器当前不可用
        - 502 和 504
          - 502 Bad Gateway — 服务器作为网关时收到**无效响应** — 可能是后端服务器崩溃
          - 504 Gateway Timeout — 服务器作为网关时，后端服务器**超时**
          - 例子 — nginx 是代理服务器，收到客户端的请求后，将请求转发到后端服务器
            - 当nginx收到了无效的响应时，就返回502
            - 当nginx超过自己配置的超时时间，还没有收到请求时，就返回504错误
    - HTTP层请求的类型
      - GET — 用于请求**获取**指定资源 — 通常用于获取数据
      - POST — 用于向服务器**提交**数据 — 通常用于提交表单数据或进行资源的创建
      - PUT — 用于向服务器**更新**指定资源
      - DELETE — 用于请求服务器**删除**指定资源
      - HEAD — 类似于GET请求，但**只返回资源的头部信息** — 用于获取资源的**元数据**而不获取实际内容
      - ❗ GET和POST
        - GET
          - GET 请求的参数位置一般是写在 URL 中
          - GET 方法就是**安全且幂等**的
            - 因为它是只读操作，无论操作多少次，服务器上的数据都是安全的
            - 可以对 GET 请求的数据做缓存，这个缓存可以做到浏览器本身上，也可以做到代理上，而且在浏览器中 GET 请求可以保存为书签
        - POST
          - POST 请求携带数据的位置一般是写在报文 body 中
          - 是不安全的，不是幂等
            - 因为是新增或提交数据的操作，会**修改**服务器上的资源
            - 浏览器一般不会缓存 POST 请求，也不能把 POST 请求保存为书签
    - ❗ HTTP的长连接
      - 定义 — 在同一个 TCP 连接上 可以发送多个 HTTP 请求，而不需要每次请求都重新建立 TCP 连接
      - 工作原理
        - 客户端发送请求 — 通过 Connection: **keep-alive** 头部，告诉服务器保持 TCP 连接不断开
        - 服务器响应请求 — 服务器收到请求后，也返回 Connection: keep-alive，并保持连接
        - 同一连接复用 — 在同一个 TCP 连接内，客户端可以继续发送多个 HTTP 请求，直到超时或主动关闭连接
    - HTTP默认的端口
      - http — 80
      - https — 443
      - DNS的端口 — 53
    - HTTP1.1怎么对请求做拆包 — 通过"Content-Length"头字段 — 该字段指示了请求正文的长度 — 服务器可以根据该长度来正确接收和解析请求
    - **HTTP为什么不安全**
      - HTTP 由于是明文传输 — 三个风险
        - 窃听风险 — 比如通信链路上可以获取通信内容，用户号容易没
        - 篡改风险 — 比如强制植入垃圾广告，视觉污染，用户眼容易瞎
        - 冒充风险 — 比如冒充淘宝网站，用户钱容易没
      - HTTPS — 在 HTTP 与 TCP 层之间加入了 SSL/TLS 协议
        - 信息加密
        - 校验机制
        - 身份证书
      - ❗ **HTTP和HTTPS 的区别**
        - HTTP
          - 是超文本传输协议，信息是**明文传输**，存在安全风险的问题
          - TCP 三次握手之后便可进行 HTTP 的报文传输
          - HTTP 默认端口号是 80
        - HTTPS
          - 在 应用层（HTTP）和传输层（TCP）之间加入了 **SSL/TLS 安全协议**
          - 在 TCP 三次握手之后，还需进行 **SSL/TLS 的握手过程**，才可进入加密报文传输
          - HTTPS 默认端口号是 443
          - HTTPS 协议需要向 CA（证书权威机构）申请数字证书 — 来保证服务器的身份是可信的
      - TLS 握手
        - 使用 **RSA 算法来实现密钥交换**的
          - 将 TLS 证书部署服务端时，**证书文件其实就是服务端的公钥**
          - 客户端会**生成随机密钥** — **并使用服务端的公钥加密**后再传给服务端
          - 根据非对称加密算法，公钥加密的消息仅能通过**私钥解密**，这样**服务端解密**后，双方就**得到了相同的密钥**，再用它**加密应用消息**
        - 过程
          - TLS 第一次握手 — 由客户端向服务器**发起加密通信请求** — 也就是 ClientHello 请求 — 发送以下信息：
            - 客户端支持的 TLS 协议版本
            - 客户端生产的随机数（Client Random） — 后面用于生成「会话秘钥」条件之一
            - 客户端支持的密码套件列表 — 如 RSA 加密算法
          - TLS 第二次握手 — 服务器收到客户端请求后，向客户端发出响应 — 也就是 SeverHello — 回应的内容：
            - 确认 TLS 协议版本 — 如果浏览器不支持，则关闭加密通信
            - 服务器生产的随机数（Server Random） — 后面用于生产「会话秘钥」条件之一
            - 确认的密码套件列表 — 如 RSA 加密算法
            - 服务器的数字证书
          - TLS 第三次握手 — 客户端收到服务器的回应之后 — 首先通过浏览器或者操作系统中的 CA 公钥，确认服务器的数字证书的真实性 — 如果证书没有问题 — 客户端会从**数字证书中取出服务器的公钥**，然后使用它加密报文 — 发送以下信息：
            - 一个随机数（pre-master key） — 该随机数会被服务器公钥加密
            - 加密通信算法改变通知 — 表示随后的信息都将用「会话秘钥」加密通信
            - 客户端握手结束通知
              - 表示客户端的握手阶段已经结束
              - 这一项同时把之前所有内容的发生的数据做个摘要，用来供服务端校验
          - 服务器和客户端有了这**三个随机数**（Client Random、Server Random、pre-master key），接着就用双方协商的**加密算法**，各自生成本次通信的「**会话秘钥**」
          - TLS 第四次握手 — 服务器收到客户端的**第三个随机数**（pre-master key）之后，通过**协商的加密算法**，计算出本次通信的「**会话秘钥**」 — 向客户端发送最后的信息
            - 加密通信算法改变通知 — 表示随后的信息都将用「会话秘钥」加密通信
            - 服务器握手结束通知
              - 表示服务器的握手阶段已经结束
              - 这一项同时把之前所有内容的发生的数据做个摘要，用来供客户端校验
      - **HTTPS是如何防范中间人的攻击**
        - 核心 — 中间人攻击的关键在于攻击者**冒充服务器与客户端建立连接，并同时与服务器建立连接**
          - 由于攻击者无法获得服务器的私钥 — 因此无法正确解密客户端发送的加密数据
          - 客户端会在建立连接时验证服务器的证书 — 如果证书验证失败或存在问题，客户端会发出警告或中止连接
        - 方法
          - 加密 — 通过非对称加密的方式来协商出对称加密密钥
          - 身份校验 — 服务器会向证书颁发机构申请数字证书
            - 证书中包含了服务器的公钥
            - 客户端会验证证书的合法性 — 如果验证通过，客户端会使用证书中的公钥来加密通信数据
    - **HTTP进行TCP连接之后，在什么情况下会中断**
      - 当服务端或者客户端**执行 close 系统调用**的时候 — 会发送FIN报文，就会进行四次挥手的过程
      - 当发送方发送了数据之后，接收方超过一段时间**没有响应ACK报文** — 发送方**重传数据达到最大次数**的时候，就会断开TCP连接
      - 当HTTP**长时间没有进行请求和响应**的时候 — 超过一定的时间，就会释放连接
    - http **断点重传**是什么
      - HTTP/1.1协议支持的特性
      - 需要**客户端记录下当前的下载进度** — 并在需要续传的时候通知服务端本次需要下载的内容片段
      - 流程
        - 客户端开始下载一个1024K的文件，服务端发送Accept-Ranges: bytes来告诉客户端 — 其支持Range的请求
        - 假如客户端下载时网络断开重连了，再下载时 — 需要在HTTP头中申明本次需要续传的片段：Range:bytes=512000 — 这个头通知服务端从文件的512K位置开始传输文件
        - 服务端收到**断点续传请求**，从文件的512K位置开始传输 — 并且在HTTP头中增加：Content-Range:bytes 512000-/1024000,Content-Length: 512000
          - 此时服务端返回的HTTP状态码应该是206 Partial Content
          - 如果客户端传过来的Range**超过资源的大小**,则响应**416** Requested Range Not Satisfiable
    - **Http1.1和2.0**
      - 头部压缩 — HTTP/2 会压缩头 — 如果你同时发出多个请求，他们的头是一样的或是相似的，那么，协议会帮你消除重复的部分 — 这就是所谓的 HPACK 算法 — 在客户端和服务器同时维护一张**头信息表**
        - 所有字段都会存入这个表，**生成一个索引号**
        - 以后就不发送同样字段了，只发送索引号，这样就提高速度了
        - 通过索引号从头信息表中获取头信息
      - 二进制格式 — HTTP/2 不再像 **HTTP/1.1 里的纯文本形式**的报文，而是全面采用了**二进制格式**
        - 头信息和数据体都是二进制，并且统称为帧（frame）
          - 头信息帧（Headers Frame）
          - 数据帧（Data Frame）
        - 虽然对人不友好，但是对计算机非常友好 — 计算机只懂二进制，那么收到报文后，无需再将明文的报文转成二进制，而是直接解析二进制报文，这增加了数据传输的效率
      - 并发传输 — 引出了 Stream 概念 — 多个 Stream 复用在一条 TCP 连接 — 解决了**HTTP/1.1 队头阻塞**的问题
      - 服务器主动推送资源 — 服务端不再是被动地响应，可以主动向客户端**发送消息** — 在一定程度上改善了传统的「请求 - 应答」工作模式
    - HTTP、SOCKET和TCP的区别
      - HTTP
        - 层级：应用层协议
        - 定义 Web 数据传输的规则
      - Socket
        - 层级：应用层接口
        - 用于描述通信链路的一端，提供了底层的通信接口
      - TCP
        - 层级：传输层协议
        - 负责在通信的两端之间建立可靠的数据传输连接
    - ❗ HTTP到底是不是无状态的
      - HTTP是无状态的
        - **每个请求都是独立的** — 服务器不会在多个请求之间保留关于客户端状态的信息
        - 通过一些机制来实现状态保持 — 最常见的方式是使用Cookie和Session来跟踪用户状态
      - 携带Cookie的HTTP请求是有状态还是无状态的 — 可以在一定程度上实现状态保持
      - Cookie是HTTP协议簇的一部分，那为什么还说HTTP是无状态的 — 虽然Cookie是HTTP协议簇的一部分
        - 但是HTTP协议在**设计初衷上仍然保持无状态特性**
        - 使用Cookie只是在无状态协议下的一种补充机制
    - HTTP长连接与WebSocket
      - HTTP长连接 — 半双工 — 每次通信仍然是单向的，即客户端请求，服务器响应
      - WebSocket — 支持全双工 — 允许双方（客户端和服务器）同时发送和接收消息
    - **session, cookie, localStorage, token**
      - session
        - 可以理解为一个状态列表
        - 数据存储在**服务器端** — 服务器为每个用户分配一个唯一的Session ID — 这个ID通常**通过Cookie或URL重写**的方式发送给客户端 — 客户端后续的请求会带上这个Session ID，服务器根据ID查找对应的Session数据
        - 主要受限于服务器的内存大小
        - Session通常认为比Cookie更安全，因为敏感数据存储在服务器端
        - 在默认情况下，当用户关闭浏览器时，Session结束
        - 每次请求都需要查询服务器上的Session数据，这可能会**增加服务器的负载**
      - cookie
        - 类似一个令牌 — 装有sessionId，存储在客户端
        - 存储在**客户端**（通常是浏览器） — 当浏览器向服务器发送请求时，会自动附带Cookie中的数据。
        - 单个Cookie的大小限制通常在4KB左右
        - Cookie相对不安全，因为数据存储在客户端，容易受到XSS（跨站脚本攻击）的威胁
        - Cookie**可以设置过期时间**，过期后自动删除 — 也可以设置为会话Cookie，即浏览器关闭时自动删除
        - 使用Cookie时，因为数据随每个请求发送到服务器，可能会**影响网络传输效率**
      - localStorage
        - 存储在**客户端**
        - 存储容量通常较大,一般限制在几 MB 左右
        - localStorage 的数据不会自动发送到服务器,它仅在浏览器端存储数据 — 适合用于在同一域名下的不同页面之间共享数据,
        - 永久存储在浏览器中,除非通过 JavaScript 代码手动删除
        - 数据仅在浏览器端存储,不会自动发送到服务器,相对而言更安全一些
      - token — 类似一个令牌，无状态 — 用户信息都被加密到token中，服务器收到token后解密就可知道是哪个用户
    - 如果客户端禁用了cookie，session还能用吗?
      - 默认情况下禁用 Cookie 后，Session 是无法正常使用的
      - 有几种方法可以绕过这个问题
        - URL重写
          - 每当服务器响应需要保持状态的请求时，将Session ID附加到URL中作为参数
          - 缺点是 — 分享了这样的链接，可能导致Session ID的意外泄露
        - 隐藏表单字段
          - 在每个需要Session信息的HTML表单中包含一个隐藏字段，用来存储Session ID
          - 缺点是 — 仅适用于通过表单提交的交互模式，不适合链接点击或Ajax请求
    - JWT 令牌和传统方式
      - 无状态性 — JWT是无状态的令牌，不需要在服务器端存储会话信息 — JWT令牌中包含了所有必要的信息，如用户身份、权限等
      - 安全性 — JWT使用密钥对令牌进行签名，确保令牌的完整性和真实性 — 只有持有正确密钥的服务器才能对令牌进行验证和解析
      - 跨域支持 — JWT令牌可以在不同域之间传递，适用于跨域访问的场景 — 通过在请求的头部或参数中携带JWT令牌
    - ❗ JWT 令牌都有哪些字段
      - 头部（Header）
        - 签名算法
        - 令牌类型
        - Base64编码进行序列化
      - 载荷（Payload）
        - 主要包含用户身份信息和声明
        - Base64编码进行序列化
      - 签名（Signature） — 对头部、载荷和密钥进行签名后的结果
    - JWT 令牌为什么能解决集群部署
      - 在集群部署中，不同服务器之间没有共享的会话信息，这会导致用户在不同服务器之间切换时需要重新登录
      - JWT令牌通过在令牌中包含所有必要的身份验证和会话信息，使得服务器无需存储会话信息，从而解决了集群部署中的身份验证和会话管理问题 — 由于JWT令牌是自包含的，服务器可以独立地对令牌进行验证，而不需要依赖其他服务器或共享存储
    - JWT的缺点
      - JWT 一旦派发出去，在失效之前都是有效的，没办法即使撤销JWT
      - 解决方法
        - 增加黑名单机制 — 使用内存数据库比如 Redis 维护一个黑名单，如果想让某个 JWT 失效的话就直接将这个 JWT 加入到 黑名单 即可 — 每次使用 JWT 进行请求的话都会先判断这个 JWT 是否存在于黑名单中
        - 及时失效令牌 — 当检测到JWT令牌泄露或存在风险时，可以立即将令牌标记为失效状态
        - 刷新令牌 — 当检测到令牌泄露时，可以主动刷新令牌，即重新生成一个新的令牌，并将旧令牌标记为失效状态
    - 前端是如何存储JWT的?
      - 不使用JWT
        - 一般流程
          - 用户向服务器发送用户名和密码
          - 服务器验证通过后，在当前对话（session）里面保存相关数据，比如用户角色、登录时间等等
          - 服务器向用户返回一个 session_id，写入用户的 Cookie
          - 用户随后的每一次请求，都会通过 Cookie，将 session_id 传回服务器
          - 服务器收到 session_id，找到前期保存的数据，由此得知用户的身份
        - 问题
          - 扩展性（scaling）不好
          - 如果是服务器集群，或者是跨域的服务导向架构，就要求 session 数据共享，每台服务器都能够读取 session
      - 使用JWT — 客户端收到服务器返回的 JWT，可以储存在 Local Storage 里面，也可以储存在Cookie里面，还可以存储在Session Storage里面
        - Local Storage（本地存储）
          - 优点
            - 提供了较大的存储空间
            - 不会随着HTTP请求一起发送到服务器，因此不会出现在HTTP缓存或日志中
          - 缺点 — 存在XSS（跨站脚本攻击）的风险 — 恶意脚本可以通过JavaScript访问到存储在Local Storage中的JWT，从而盗取用户凭证
        - Session Storage（会话存储）
          - 优点 — 与Local Storage类似，但仅限于当前浏览器窗口或标签页 — 当窗口关闭后数据会被清除，这在一定程度上减少了数据泄露的风险
          - 缺点 — 用户体验可能受影响，因为刷新页面或在新标签页打开相同应用时需要重新认证
        - Cookie
          - 优点
            - 可以设置HttpOnly标志来防止通过JavaScript访问，减少XSS攻击的风险
            - 可以利用Secure标志确保仅通过HTTPS发送，增加安全性
          - 缺点
            - 大小限制较小（通常4KB）
            - 并且每次HTTP请求都会携带Cookie，可能影响性能
  - Nginx
    - Nginx在应用层
    - 负载均衡算法
      - 轮询 — 按照**顺序依次将请求分配**给后端服务器
      - IP哈希 — 根据**客户端IP地址的哈希值**来确定**分配请求的后端服务器** — 适用于需要保持同一客户端的请求始终发送到同一台后端服务器的场景
      - URL哈希 — 按访问的URL的哈希结果来分配请求 — 使每个URL定向到一台后端服务器，可以进一步提高后端缓存服务器的效率
      - 最短响应时间 — 按照后端服务器的响应时间来分配请求，响应时间短的优先分配
      - 加权轮询 — 按照权重分配请求给后端服务器，权重越高的服务器获得更多的请求

Presented with xmind